

FastAPI + WebSocket ve Web Panel Mimarisi

Bu düzeltilmiş sürüm, sağlanan PDF ve ilk düzeltme notunu ve ikinci turdaki yedi maddeyi temel alır. Mimari sözleşmesini tanımlar; uygulama kodunun değiştirilmiş veya çalıştırılarak doğrulanmış olduğunu ifade etmez. Dört belge aynı config, zaman ve API kararlarını kullanır.

1. REST komut/sorgu, WebSocket canlı akış

React -> REST -> FastAPI -> Application -> Command Queue
ROS callbacks -> Event Queue -> Tek Fleet Manager consumer
Consumer -> SQLite + outbox -> immutable read model
Read model -> REST snapshot / WebSocket olay + telemetri

Fleet Manager robot, görev ve filo stateinin tek sahibidir. API ve ROS callbackleri doğrudan state yazmaz. Application use-case katmanıdır; UI değildir. Queue kabulü komutun tamamlanması değildir: 202 + command_id alınır; sonuç command sorgusu ve task_update/fleet_event ile izlenir.

2. Deployment ve erişim sınırı

MVP tek FastAPI worker ve tek uygulama replica kullanır. Bellek replay ve snapshot çok process arasında paylaşılmaz. Dağıtım/README sözleşmesi: uvicorn <uygulama_modülü>:app --workers 1. Çok worker veya çok replica bu mimaride desteklenmez.

Kapsam yerel ağ/demo: kullanıcı tokenı POST /api/session gövdesinde bir kez sunar. Sunucu doğrulayıp opak oturum kimliğini HttpOnly cookie olarak kurar. REST ve WebSocket aynı cookieyi kullanır; SPA cookieyi okumaz, Bearer veya query token kullanmaz.

POST /api/session dahil mutasyonlarda ve WebSocket handshake sırasında Origin allowlist doğrulanır. /api/session dışında REST ve WS geçerli oturum ister. Token deployment secret/env değeridir; loglanmaz. HttpOnly, SameSite=Strict, Path=/ ve TLS üzerinde Secure cookie kullanılır. Ayrıntılı oturum akışı son bölümdedir.

Process bellek sınırı: <https://fastapi.tiangolo.com/deployment/concepts/> . Bu belgenin kalan protokol ayrıntıları projeye özgü tasarım sözleşmesidir.

REST uçları ve hata biçimi

Uç	Davranış
GET /api/fleet/status	Tutarlı snapshot; stream_id, sequence ve config revisionlar.
GET /api/robots/{id}	Robot listesi veya detay/istatistik.
GET /api/tasks/{id}	Görev listesi/detayı; state ve robot filtresi.
POST /api/tasks	request_id veya Idempotency-Key; 202, task_id + command_id.
POST /api/tasks/{id}/cancel	202; pickup sonrası / yük belirsizliğinde 409. DELETE iptal ucu yok.
POST /api/robots/{id}/resolve	202; operatör çözüm kaydı, payload teyidi; güvenli uzlaştırma şart.
GET /api/commands/{id}	Komut durumu: pending/succeeded/failed + sonuç/hata.
GET /api/map	image_url, resolution (m/pixel), origin {x,y,yaw}, frame_id, map_revision.
GET /api/locations	id, name, kind (pickup/dropoff/charger), pose, frame_id; istasyon kapasitesi.
GET /api/experiments/{id}	Salt okunur sonuç listesi veya koşu detayı/metrikler.

Görev oluşturma pickup/dropoff konumu veya location_id, priority ve istemci tekrar önleme anahtarı alır. İki anahtar birlikte verilirse eşit olmalıdır. Aynı anahtar + aynı normalize gövde aynı sonucu; aynı anahtar + farklı gövde 409 döndürür. command_id bu anahtarın yerine geçmez.

Cancel ve resolve kabulü consumer içinde tekrar doğrulanır. Resolve; reason, payload_resolution ve expected_entity_version ister. Eski goal kapanmamış, telemetri bayat veya payload çözülmemişse başarılı olamaz; robot koşullar doğrulanmadan IDLE yapılmaz.

```
{
  "error": {
    "code": "revision_conflict",
    "field": "If-Match",
    "message": "Taslak değişti; yeniden yükleyin.",
    "current_revision": "r18"
  }
}
```

404 bulunamadı; 409 durum/kimlik/idempotency çatışması; 412 eski If-Match; 422 alan doğrulama; 428 eksik If-Match; 401/403 kimlik/erişim; 503 servis/kapasite sorunu. Liste sorguları cursor + limit ile sayfanır; snapshot istemcinin takip ettiği güncel varlıkları tam içerir.

Config, Apply ve görünürlük

Uç	Sözleşme
GET /api/config	Taslak, ETag, active/pending/running_revision ve alan uygulama sınıfları.
PATCH /api/config	Skaler/nesne ayarları; robots değişikliği 422.
GET /api/config/robots	Taslak liste.
POST /api/config/robots	201; taslağa ekle.
PUT /api/config/robots/{id}	200; tam tanım, id değişmez.
DELETE /api/config/robots/{id}	204; yalnız taslak, geçmiş soft delete ile korunur.
POST /api/config/apply	202 + operation_id; bütün taslağı doğrula ve etkinleştir.
GET /api/config/operations/{id}	Apply aşaması, status, revisionlar, hata.

Mutasyonlar If-Match: "<pending_revision>" alır. CRUD ve PATCH yalnız bellekteki taslağı değiştirir. Save çalışan filoyu değiştirmez. Apply tek fleet.yaml dosyasını aynı dizinde temp + os.replace ile kaydeder. active kalıcı hedef, pending taslak, running başarıyla çalışan sürümdür. Changes pending üç revision eşit değilse gösterilir.

Live alanlar: strategy/weights, beş batarya politika alanı, görev süreleri/retry, navigation_timeout. Yeni karar/aşama/goal/şarj oturumunda geçer; aktif timer/retry/şarj oturumu geriye dönük değişmez. Restart alanları: robot seti/model/spawn, Nav2 profil/goal_tolerance, batarya modeli/katsayıları ve simülasyon ayarları.

Apply sırasında komut ve config mutasyonları 409 applying; servis geçici kullanılamıyorsa 503 applying döner. GET snapshot/işlem sorguları açık kalır. Bu koruma yalnız Restart-Apply içindir: aktif teslimat veya ONBOARD/UNKNOWN robot 409 üretir. Live Apply görev sürerken kabul edilir. Restart-Apply QUEUED görevleri korur ve registry kurulmadan manager_epoch yeniler. Yeniden başlatma sonrası hazırlık doğrulanmadan Applied gösterilmez.

fleet_event türleri: config_apply_started, config_apply_progress, config_applied, config_apply_failed. payload operation_id, phase, active/pending/running_revision ve gerekirse error taşır. Registry robot seti yenilenince resync_required gönderilir. Başarısız Applyda eski running_revision veya null görünür; active hedefle fark gizlenmez.

Snapshot ve replay protokolü

3. Açılış ve yeniden bağlanma

1. GET /api/fleet/status snapshotını al. 2. /ws/fleet?stream_id=<id>&after_sequence=<n> ile bağlan. 3. Sunucu n sonrasını replay eder, ardından aynı sırada canlı olaylara geçer. Replay/canlı geçişi tek yayın kilidi veya atomik cursor kaydıyla yapılır; arada olay kaybolmaz.

```
{"schema_version":1,"stream_id":"a3f9","sequence":1842,
"timestamp":"2026-10-09T09:30:00Z","sim_time":125.5,
"config":{"active_revision":"r18","pending_revision":"r18",
"running_revision":"r18"},
"robots":[],"tasks":[],"mode":"normal","simulation_paused":false}
```

robots/tasks bu örnekte boş filo için gösterilmiştir. Dolu filoda tam robot/görev nesneleri, entity_version ve son telemetri bulunur. Snapshot ile sequence aynı consumer kesitinden üretilir.

Sunucu son 1000 replay olayını bellek halka tamponunda tutar. Son sequence=n ve en eski tutulan sequence=m ise $m-1 \leq \text{after_sequence} \leq n$ geçerlidir. Uyumsuz stream_id, tampondan düşmüş veya gelecekteki cursor resync_required üretir. Process restartı yeni stream_id başlatır; eski sequence başka akışla karşılaştırılmaz.

```
{"type":"resync_required","schema_version":1,
"stream_id":"a3f9","timestamp":"2026-10-09T09:30:05Z",
"reason":"cursor_expired"}
```

İstemci resync mesajında eski görünümü güncel saymaz, bağlantıyı kapatır, snapshotı yeniler ve yeni cursorla bağlanır. Ayrı “eksikleri iste” mesajı yoktur. İstemci sequence $\leq$ son uygulanan değeri tekrar olarak atar; $>$ son+1 boşluksa yeniden bağlanır/replay dener. Başarısız replay snapshot gerektirir.

Replay olayları kalıcı domain değişikliklerini temsil eder, fakat halka tampon disk kalıcılığı sağlamaz. Kalıcı geçmiş SQLite journalındadır; restart sonrası WebSocket replay devamlılığı vaat edilmez. Her yeni bağlantıda replay sonrasında robot başına son telemetri de gönderilir; snapshot-bağlantı arasındaki telemetri yarışı kapanır.

Tam nesne ve geçici telemetri

4. Replay olayları

```
{
  "type": "task_update",
  "schema_version": 1,
  "stream_id": "a3f9",
  "sequence": 1843,
  "timestamp": "2026-10-09T09:30:01Z",
  "entity_id": "task_024",
  "entity_version": 7,
  "payload": {
    "id": "task_024",
    "request_id": "req_024",
    "state": "QUEUED",
    "priority": 0,
    "assigned_robot_id": null,
    "pickup_pose": {
      "x": 1,
      "y": 2,
      "yaw": 0,
      "frame_id": "map"
    },
    "dropoff_pose": {
      "x": 3,
      "y": 4,
      "yaw": 0,
      "frame_id": "map"
    },
    "payload_state": "NONE",
    "stage_retry_count": 0,
    "requeue_count": 0,
    "created_at": "2026-10-09T09:30:01Z",
    "started_at": null,
    "completed_at": null,
    "entity_version": 7
  }
}
```

task_update tam görev nesnesidir; robot_update tam ayrı RobotState nesnesidir. REST birleşik robot nesnesinin tamamı değildir. İkisi de kendi şemasında patch değil tam nesnedir. null alanı boş değerdir; eksik zorunlu alan protokol hatasıdır. Aynı stream içinde daha büyük entity_version kazanır. robot_update yalnız id, state, active_task_id, goal, payload_state, incident ve charge_phase alanlarını kapsar; entity_version yalnız bu ayrı alanlar değişince artar. Pose, SoC, battery_status ve sürekli sayaçlar bu olayda yoktur. fleet_event de sequence taşır.

5. Geçici telemetri

```
{
  "type": "robot_telemetry",
  "schema_version": 1,
  "stream_id": "a3f9",
  "timestamp": "2026-10-09T09:30:02Z",
  "entity_id": "robot_2",
  "telemetry_version": 29,
  "telemetry_at": "2026-10-09T09:30:02Z",
  "sim_time": 126.0,
  "payload": {
    "pose": {
      "x": 1.2,
      "y": 3.4,
      "yaw": 0.1,
      "frame_id": "map"
    },
    "battery_soc": 0.61,
    "battery_status": 2
  }
}
```

robot_telemetry sequence taşımaz, replay edilmez; robot başına en son ölçüm tutulur. telemetry_at sunucunun son geçerli ölçümü aldığı UTC zamandır; sim_time ayrı /clock değeridir. Pose/batarya farklı anda geliyorsa pose_at ve battery_at ayrıca taşınır; biri diğerini güncel göstermez. telemetry_version aynı robot/stream için artar ve ters sırada gelen örneği eler.

İstemci domain entity_version ve telemetry_version sayaçlarını ayrı tutar. Snapshot yüklenince ikisi de o snapshot değerine kurulur; stream değişince eski sürümler bırakılır. Simülasyon saati reseti telemetry_at değerini geriye götürmez.

Canlılık, kuyruk ve panel

6. Sınırlı kuyruk ve heartbeat

Bağlantı başına sınırlı gönderim kuyruğu vardır. Telemetry robot başına birleştirilir; dolulukta önce telemetry düşer. Replay olayları kuyruğa sığmıyorsa resync_required gönderilmeye çalışılır ve bağlantı kapatılır. Mesaj bile gönderilemiyorsa istemci kopuşta aynı resync/reconnect kuralını uygular.

```
{"type":"heartbeat","schema_version":1,"stream_id":"a3f9",
"timestamp":"2026-10-09T09:30:05Z","sequence":1843}
```

Heartbeat yeni sequence tüketmez; son üretilmiş replay sequence değerini taşır. Aynı sokette önceki olayların arkasından gönderilir. Örnek aralık 5 s, bağlantı bayatlık sınırı 15 s duvar saatidir. İstemci heartbeat sequence ilerideyse eksik olay ihtimalini anlar; snapshot/replay ile uzlaştırır. Simülasyon pause olsa da bağlantı heartbeat sürer.

7. Kullanıcı görünümüleri

Sayfa	İçerik
Dashboard	Harita, robot kartları, aktif görevler, uyarılar; map ve locations uçlarından veri.
Robot Details	SoC/status, pose/frame/zaman, current_goal, payload, hata ve kalıcı toplam istatistik.
Tasks	Oluşturma, filtre, detay, izinli aşamada cancel; request_id korunur.
Experiments	MVP yalnız sonuç listesi, koşu detayı ve strateji karşılaştırması. Başlat düğmesi yok.
Configuration	Fleet/Robots/Battery/Assignment/Navigation/Tasks/Simulation; taslak ETag, Apply ve üç revision.

Panel bağlantı durumunu, son ölçüm zamanını ve simulation_paused bilgisini gösterir. Bayat pose veya SoC soluk/uyarılı gösterilir; güncel gibi sunulmaz. Bayatlık telemetry_at/alan zamanlarından hesaplanır; sunucu timestamp ile saat farkı düzeltilir ve yerel monotonic geçen süre kullanılır.

Statistiklerde distance_travelled ham odomdan coalesce öncesi hesaplanır. completed_tasks/failed_tasks görev tablosundan türetilir. active_time ASSIGNED...UNLOADING simülasyon süresidir, şarj hariç. charging_count doğrulanmış oturumları sayar. Toplam/run ayrımı açık gösterilir; soft delete geçmişini korur.

Deneyler ve doğrulama

8. CLI/batch öncelikli deney

Girdi senaryo + seed + stratejidir. Her koşu öncesi robot pozları, SoC, görev/goal, timer ve run sayaçları sıfırlanır. Simülasyon hazır olma bariyeri geçilmeden ölçüm başlamaz. Canlı kullanıcı görev/resolve/cancel ve config mutasyonları deney kilidi boyunca 409 experiment_running alır. Runner görev komutları ve resolve için ayrı yetkili iç akış kullanır; simülasyonda yük temizleme ve duruş kanıtı doğrulanmadan robot serbest bırakılmaz.

ExperimentRun: id, scenario_id, seed, strategy, config_hash, status, started_sim_time, completed_sim_time, metrics, failure_reason. SQLite sonuçları saklar; CLI CSV üretir. GET /api/experiments cursor/limit ve durum/strateji filtresi alır; GET /api/experiments/{id} ayrıntıları verir. Webden senaryo başlatma MVP dışıdır.

Süreler /clock ile ölçülür; görev süreleri 5.0 s, max_navigation_retries=2 (toplam 3 girişim). Tüm stratejiler aynı filtre/model/başlangıç koşullarıyla karşılaştırılır. CostBased wd/we/wt/wb + opsiyonel wl kullanır; varsayılan .35/.30/.20/.15/0 ve toplam 1dir.

9. Protokol kabul kontrolleri

Senaryo	Beklenen
Snapshot ile WS arasında olay	after_sequence replay ile kayıp yok.
Yanlış stream / eski cursor	resync_required; yeni snapshot.
Tekrar / ters entity sürümü	İdempotent uygulama; eski sürüm yok sayılır.
Telemetri yoğunluğu / yavaş istemci	Telemetri coalesce; olay taşması resync/kopuş.
Apply / worker restart	Üç revision görünür; robot setinde resync, process restartında yeni stream.
İki editör / tekrar görev isteği	Eski ETag 412; aynı request_id tek görev.
Deney / pause / soft delete	Kilit, /clock süreleri, geçmiş istatistikler korunur.

Kaynak: fastapi-websocket-web-panel-mimarisi.pdf v1.0 (6 sayfa) ve düzeltme notu A1-C17. İlgili belge seti: Config v1.3 FIX; Fleet Manager v2.2 FIX; Battery Simulator v1.2 FIX. Bu sürüm ilk düzeltme turunu ve ikinci turdaki yedi maddeyi birlikte kapsar.

Alan bölüşümü ve oturum akışı

Kanal	Tam nesne sınırı / sürüm
robot_update: replay var	id, state, active_task_id, goal, payload_state, incident, charge_phase, entity_version. Yalnız ayrık değişimler entity_version artırır.
robot_telemetry: replay yok	Pose, battery_soc, battery_status; alan ölçüm zamanları ve telemetry_version. Son örnek robot başına tutulur.
REST Robot Details	Ayrık state + son telemetry + stats birleşimi. distance_travelled/active_time dahil stats REST üzerinden, stats_at ve sim_time ile okunur. Sürekli sayaçlar olay üretmez.

Task güncellemeleri tam Task nesnesi olmaya devam eder. Robot Details açıkken UI örneğin 2 s aralıkla REST sorgular; sekme gizliyen durdurur. Bu UI yenileme önerisi domain timerı değildir. REST entity_version yalnız ayrık state sürümüdür; telemetry ve istatistik tazeliği ayrıca değerlendirilir.

Tek cookie ile REST ve WebSocket

POST /api/session {token: ...}: allowlist Origin ve token doğrulanır; başarılıysa 204 + Set-Cookie: fleet_session=<opak rastgele id>; HttpOnly; SameSite=Strict; Path=/; Max-Age=28800. TLS ortamında Secure zorunludur. Yerel HTTP geliştirmede Secure kullanılmaması yalnız localhost geliştirme ayarıdır.

Oturum sunucu belleğinde 8 saat yaşar; restartta geçersizleşir. POST başarısı tokenı response gövdesinde döndürmez. Yanlış token 401, izin verilmeyen Origin 403. DELETE /api/session oturumu iptal edip cookieyi siler (204); oturum süresi dolunca REST 401, açık WS 4401 ile kapanır. Yetkisiz WS handshake reddedilir.

SPA aynı-origin REST isteklerinde cookieyi tarayıcıya taşır; JS HttpOnly değerini okuyamaz ve okumaz. Login sonrasında token uygulama belleğinden temizlenir, localStorage veya URLye yazılmaz. Durum değiştiren uçlarda Origin zorunlu ve allowlist olmalıdır; WebSocket handshake de aynı denetimi yapar.

Vite geliştirme proxy: /api HTTP ve /ws WebSocket isteklerini aynı backend hedefine iletir; WS proxy etkin olmalıdır. Tarayıcı yalnız Vite originine bağlanır. Backend allowlist geliştirmede Vite originini, dağıtımda panel originini kapsar. Cookie Domain verilmez; host-only kalır. Üretimde reverse proxy aynı-origin /api ve /ws sağlar.

Apply, toparlanma ve yeni kontroller

Yalnız Live alanlı Apply aktif teslimat varken kabul edilir; karar/goal/aşama/şarj oturumunun mevcut yakalama kuralları sürer. İşlem sırasında kısa mutasyon kilidi 409 applying döndürebilir. Restart alanı içeren Apply ise aktif teslimat veya ONBOARD/UNKNOWN varsa 409 üretir; QUEUED kayıtları korunur, registry kurulmadan manager_epoch yenilenir.

Yüksüz doğrulanmış Nav2 hatası TEMP_UNAVAILABLE olarak görünür; cooldown ve taze telemetri/Nav2 kontrolüyle otomatik döner. Resolve düğmesi normal modda ONBOARD/UNKNOWN incident çözümü içindir. Deney runnerına resolve iç yetkisi verilir; simülasyonda duruş ve yük temizliği doğrulanmadan serbest bırakılmaz. Başarısız teslimat metriği korunur.

Fırsat şarjı: IDLE ve SoC < idle_charge_level (0.50) ise veya ulaşılabilir bir bekleyen görev yalnız enerji yeterliliği nedeniyle eleniyorsa şarj planlanır. $low < idle_charge_level \leq release$ doğrulanır. Örnekte SoC=0.30 robot, 0.307 ihtiyacı karşılayamadığında $low=0.25$ eşliğine düşmeyi beklemes. İstasyon erişimi/rezervasyonu doğrulanır; kapasite yoksa charge_pending görünür kalır ve yeniden değerlendirilir. FULL robotun enerji filtresinden geçemediği iş için sonsuz şarj döngüsü kurulmaz; görev energy_infeasible olarak görünür kalır.

Yedi madde için kabul ölçütleri

Kontrol	Beklenen
SoC 0.30, ihtiyaç 0.307	Beklemek yerine fırsat şarjı planlanır.
10 Hz pose/SoC ve artan stats	Replay buffer yalnız ayırık robot olaylarıyla büyür.
Aktif teslimat + Live strategy	Apply kabul; aktif görevi restart etmez.
Restart + QUEUED görev	Görev korunur; yeni epoch eski sonucu eler.
Yüksüz retry tükenmesi	Operatörsüz cooldown ve hazır kontrolüyle dönüş.
ONBOARD deney hatası	FAILED metriği + teyitli simülasyon resolve; başarıya çevrilmez.

Şema / topic / auth

Eksik alanlar doğrulanır; pose/contact üreticileri bellidir; login cookieyle REST ve WS açar.

Oturum/proxy referansları: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Set-Cookie> ;
<https://vite.dev/config/server-options.html#server-proxy>

İkinci tur kaynak: kullanıcının yedi maddelik düzeltme isteği. Bu tasarım güncellemesi uygulama kodunda veya çalışan serviste test yapılmış olduğu anlamına gelmez.